

Session 4 · Deck 4.B5

Worked Example: The Stochastic RBC Model, Properly

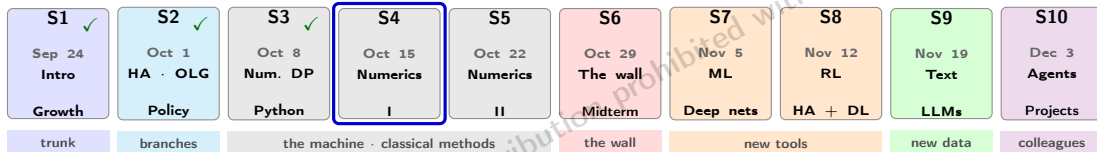
Zhigang Feng

Thursday, October 15, 2026

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are

you are here: 4.B5 ▼



Build it on paper → solve it on the machine → push it to the frontier with AI

Block B: 4.B1 approximation and interpolation · 4.B2 quadrature · 4.B3 discretizing an AR(1) · 4.B4 fast and accurate · 4.B5 the worked example, end to end.

The Same Deck as 3.A4, With the Tools Turned On

In 3.A4 every one of the six decisions was made the crudest way: a hand-written two-state chain, no interpolation, a grid search for the maximum, and a full sweep every iteration. We re-run *that solver* here, at this session's quarterly calibration, to get an honest "before" column.

Then the same five steps, with this session's replacements in place:

3.A4	here
Π by hand	Rouwenhorst (4.B3)
no interpolation	linear , which keeps concavity (4.B1)
grid search	golden section (4.A2)
full sweeps	Howard , $n_H = 20$ (4.B4)

The model

$$v(k, z) = \max_{k'} \left\{ \ln c + \beta \mathbb{E}[v(k', z') \mid z] \right\}$$

$$c + k' = zk^\alpha + (1 - \delta)k, \quad \ln z' = \rho \ln z + \sigma \varepsilon'.$$

$$\text{Quarterly: } \alpha = 0.33, \beta = 0.99, \delta = 0.025, \\ \rho = 0.95, \sigma = 0.007.$$

No closed-form policy. The benchmarks are the steady state, the chain's own moments, and the Euler residual.

Step 1: The Chain, and Checking It Before Anything Else

```
def rouwenhorst(n, rho, sigma):  
    p = (1 + rho) / 2  
    P = np.array([[p, 1-p], [1-p, p]])  
    for k in range(3, n+1):  
        Z = np.zeros((k, k))  
        Z[:-1, :-1] += p*P;      Z[:-1, 1:] += (1-p)*P  
        Z[1:, :-1] += (1-p)*P; Z[1:, 1:] += p*P  
        Z[1:-1, :] /= 2.0  
        P = Z  
    sz = sigma / np.sqrt(1 - rho**2)  
    psi = sz * np.sqrt(n - 1)  
    return np.linspace(-psi, psi, n), P  
  
logz, Pi = rouwenhorst(7, 0.95, 0.007)  
z = np.exp(logz)
```

implied ρ	0.950000
implied σ_z	0.02241794
target σ_z	0.02241794
z range	[0.947, 1.056]

Check the chain before you solve anything.

Three lines, and it rules out the single most common silent error in this literature — a shock process that is not the one you wrote down.

Step 2: The Benchmark, and the Grid

```
alpha, beta, delta = 0.33, 0.99, 0.025

# the one thing we know exactly
kss = ((1/beta - 1 + delta)/alpha)**(1/(alpha-1))
# 28.348419061048443

n_k = 300
kg = np.linspace(0.5*kss, 1.5*kss, n_k)
h = kg[1] - kg[0] # 0.0948

# cash on hand, every (state, shock)
y = (z[None,:] * kg[:,None]**alpha
     + (1-delta)*kg[:,None])
```

$k^* = 28.3484$, $y^* = 3.0153$, $c^* = 2.3066$, and $K/Y = 9.40$ at a quarterly frequency — about 2.35 annual, which is the number to sanity-check against.

The grid is $\pm 50\%$ around k^* . Print $\max_i g(k_i)/k_N$ when you are done: if it is 1, the policy is pinned at the top of the grid and the answer is censored, not solved.

Step 3: The Inner Maximization Is No Longer a Search

```
PHI = (np.sqrt(5) - 1) / 2

def rhs(kp, yj, EVj):    # Bellman RHS, off grid
    c = np.maximum(yj - kp, 1e-12)    # clamp
    return np.log(c) + beta*np.interp(kp,kg,EVj)

# golden section, vectorised over all k at once
yj, EVj = y[:, j], EV[:, j]
a = np.full(n_k, kg[0])
b = np.minimum(yj - 1e-8, kg[-1])
c_, d_ = b - PHI*(b-a), a + PHI*(b-a)
fc, fd = rhs(c_, yj, EVj), rhs(d_, yj, EVj)
for _ in range(40):
    L = fc > fd
    b = np.where(L, d_, b); a = np.where(L, a, c_)
    c_, d_ = b - PHI*(b-a), a + PHI*(b-a)
    fc, fd = rhs(c_, yj, EVj), rhs(d_, yj, EVj)
g[:, j] = 0.5*(a + b)
```

Three of this session's pieces, in one block:

- `np.interp` is **4.B1's** linear interpolation — chosen because it keeps $\hat{v}$ concave;
- that concavity is what makes the objective **unimodal**;
- which is what licenses **4.A2's** golden section to shrink the bracket safely.

Swap in a cubic spline and the first bullet fails, so the third one does too. **The interpolation choice is not cosmetic** — it is a hypothesis of the maximizer.

Step 4: The Expectation, and Howard Around the Outside

```
V = np.zeros((n_k, n_z)); Vn = np.empty_like(V)
while True:
    EV = V @ Pi.T          # (n_k, n_z): one matmul
    for j in range(n_z):
        g[:, j] = golden_section(j, EV)
        Vn[:, j] = rhs(g[:, j], y[:, j], EV[:, j])
    d = np.max(np.abs(Vn - V))
    V[:] = Vn              # copy, do not rebind

    for _ in range(20):    # Howard: no argmax
        EV = V @ Pi.T
        for j in range(n_z):
            V[:, j] = rhs(g[:, j], y[:, j], EV[:, j])
    if d < 1e-6:
        break
```

$V @ \text{Pi.T}$ is the whole expectation — $\mathbb{E}[v(k', z') \mid z]$ for every (k', z) at once. It is computed **once per sweep**, outside the maximization, not re-summed inside it.

The inner block is Howard: the policy g is *held fixed* and only the value is updated, so there is no `argmax` and no bracket — just an interpolation and an add.

Two nested loops, and they do completely different work. Confusing them is the most common way this algorithm is implemented wrongly.

Step 5: What It Bought, Stage by Stage

Same model, same grid ($n_k = 300$), same chain, same tolerance — only the inner maximization and the sweep structure differ.

	3.A4 style	this deck
policy updates	1,352	66
wall time	1.03 s	0.93 s
inner work	852M comparisons	11.4M evals
worst $\log_{10} \mathcal{E}$	-1.30	-2.52
mean $\log_{10} \mathcal{E}$	-2.14	-3.40
distinct $k' / 36$ states	35	36

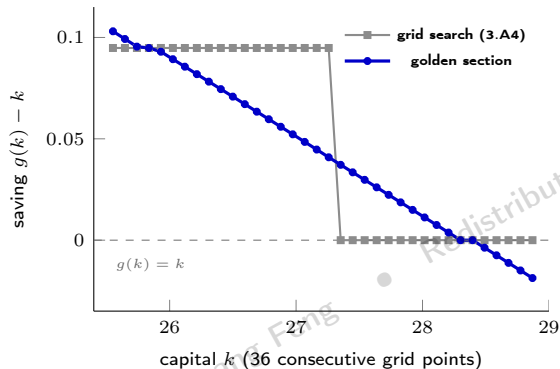
1.2 decades of accuracy, a twentieth of the policy updates, and inner work down by a factor of 75 — at slightly *less* wall clock. 4.A2 and 4.B4 report the same experiment at $n_k = 500$ (-1.55 and -2.77); the errors there are smaller because the grid is finer. The gain is 1.2 decades either way.

Where each gain came from

The **iteration count** is Howard. The **accuracy** is golden section plus interpolation. The **chain** changed nothing about speed and everything about what the model is.

But do not read too much into the clock. The $75\times$ drop in evaluations buys only a 10% drop in time, because golden section runs in a Python loop while the grid search is one array operation. On 4.B4's finer grid the same comparison runs *slower*. Report the operation count, which is implementation-independent, *and* the clock.

The Staircase, Retired

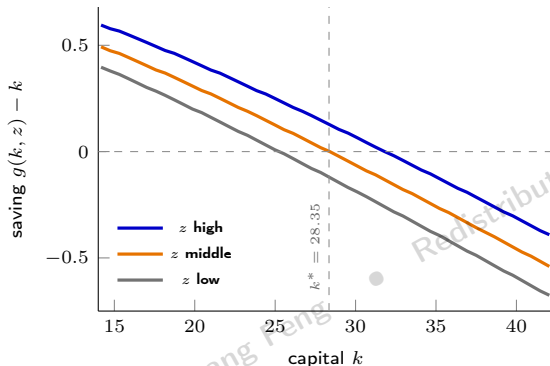


Thirty-six consecutive grid points in the middle state, plotted as *saving* rather than k' — the same data, with the 45-degree line subtracted off.

- Grid search can only pick $k' = k + mh$, so saving takes **two values in this whole window**: one grid spacing, then zero. It is a staircase with two steps.
- Golden section gives a **smooth, declining** line that crosses zero once.

And the two cross zero a full **ten grid spacings apart** — 27.35 against $k^* = 28.35$. **4.A2** had to make this point with the residual, because the plot of k' against k hides it entirely. Subtract the 45-degree line and it is unmissable.

The Policy, and the Three Checks



Three things to look at, in this order:

1. The three curves are **ordered in z** and do not touch — a good shock saves more at every k . A transposed Π breaks this immediately.
2. Saving is **declining** in k and nearly linear, which is the concavity the theory promises.
3. The middle curve crosses zero at $k^* = 28.35$; the low and high curves cross on either side. Those are the model's conditional steady states.

Note how *shallow* these curves are — saving moves by about 1 unit while k moves by 28. That is why the previous slide's grid-search crossing missed by ten spacings: a flat function makes a weak test. **Use the residual to measure accuracy**; use the picture for sign checks only.

What Is Still Crude

Even with the whole Session 4 toolkit switched on, three things in this code are still the cheap option:

- **Linear interpolation.** $O(h^2)$, and the kinked derivative is felt by anything that differentiates $\hat{v}$. A shape-preserving spline would do better without losing concavity.
- **One continuous state.** Add labour or a second asset and the grid is n_k^2 — and 4.B1's **Smolyak** is the first answer, S5's projection the second.
- **We still iterate on the value function.** 4.B4's EGM skips it entirely: $10^{-6.06}$ against this solver's $10^{-2.77}$ on the same grid — **three further decades**, and 38 times faster.

The honest summary

This is a good solver. It is not the best one in the session — EGM is — and the reason to write this one first is that **every piece of it generalises** to models where EGM does not apply.

EGM needs an invertible u' , one continuous control and a monotone policy. Value iteration with golden section needs none of those, which is why it is still the workhorse for models with discrete choices or binding constraints.

Takeaways

- The same five steps as 3.A4, with four substitutions: **Rouwenhorst, interpolation, golden section, Howard**. The code is barely longer.
- **Check the chain first**. Implied $\rho = 0.950000$ and σ_z matching the target to eight digits — three lines that rule out the commonest silent error in the literature.
- The interpolation choice is a **hypothesis of the maximizer**: linear keeps $\hat{v}$ concave, concavity gives unimodality, and unimodality is what licenses golden section. A cubic spline breaks the chain at the first link.
- Measured: **66 policy updates against 1,352**, inner work down $75\times$, and **1.2 decades** of Euler accuracy. The wall clock barely moves, and its sign flips with the grid size — which is why you report the operation count too.
- It is still not the best solver here. **EGM is**, by three further decades — but this one survives into models with discrete choices and binding constraints, where EGM cannot go.

Bridge: Two Sessions, One Model, Many Solvers

Sessions 3 and 4 have now solved the same model many different ways — grid VFI, time iteration, Howard, monotonicity, continuous choice, EGM — and every difference between them was measured rather than asserted. That is the habit the rest of the course rests on.

What has not changed is the **representation**: values, on a grid, one number per state. Everything we have built makes that representation cheaper or more accurate. None of it makes the grid go away — and at two continuous states it already hurts.

Next week (S5)

Perturbation discards the grid and expands around the steady state. **Projection** replaces N separate values with a handful of basis coefficients — and Smolyak returns, now as the basis rather than the node set.

Then this same model is solved both ways, and put beside today's.

Homework 1 is due October 22. Part B Q3 is this deck's model, and asks you to build the chain both ways and see whether it moved the answer.